

Projektna dokumentacija

CRM/HRM mikroservisni sistem

Repozitorijum: <https://github.com/LukaC011/devOps>

Integracija razvoja i operative u informacionim tehnologijama

Luka Davidović

Broj indeksa: IT51-2022

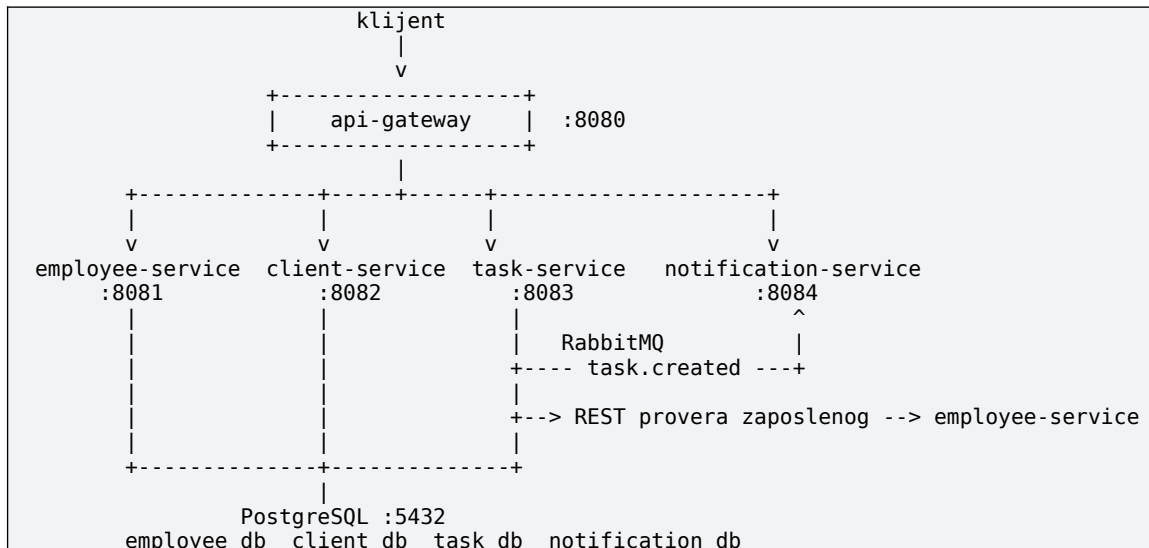
Avgust 2026.

Sadržaj

1. Opis sistema i arhitektura
2. Tehnologije i obrazloženje izbora
3. Ispunjenost zahteva po tačkama specifikacije
4. Tabela: zahtev do implementacije
5. Git workflow i istorija pull request-ova
6. Uputstvo za lokalni deploy
7. Dokazi o radu sistema

1. Opis sistema i arhitektura

Sistem je CRM/HRM aplikacija podeljena na pet mikroservisa. Poslovni domen obuhvata zaposlene, klijente, zadatke koji se dodeljuju zaposlenima i notifikacije koje nastaju kao posledica kreiranja zadatka.



Arhitektura sistema. Gateway je jedina ulazna tačka, svaki servis ima svoju bazu.

Ključne arhitektonske odluke

Gateway kao jedina ulazna tačka. Svi poslovni pozivi idu kroz api-gateway na portu 8080, sa prefiksom /api. Gateway rutira po Path predikatu i skida prefiks StripPrefix filterom, tako da servisi iza njega vide svoje originalne putanje.

Baza po servisu. Umesto jedne deljene šeme, svaki servis ima svoju bazu unutar iste PostgreSQL instance. Baze se kreiraju automatski, init skriptom montiranom u /docker-entrypoint-initdb.d/. Nijedan servis ne čita tuđe tabele, komunikacija ide isključivo kroz REST ili poruke.

```
CREATE DATABASE employee_db;
CREATE DATABASE client_db;
CREATE DATABASE task_db;
CREATE DATABASE notification_db;
```

initdb/01-create-databases.sql

Konfiguracija kroz environment promenljive. Nijedan port, URL servisa, kredencijal baze ili adresa brokera nije hardkodovan. Sve dolazi iz .env fajla kroz Docker Compose, sa razumnim podrazumevanim vrednostima za pokretanje van kontejnera.

Tok jednog zahteva

Kreiranje zadatka je scenario koji dodiruje sve tipove komunikacije:

1. Klijent šalje POST /api/tasks gateway-u.
2. Gateway rutira zahtev na task-service.
3. task-service sinhrono, preko REST-a, pita employee-service da li zaposleni postoji. Ako ne postoji, lanac se prekida sa 404.
4. task-service upisuje zadatak u svoju bazu.
5. task-service objavljuje task.created događaj na RabbitMQ topic exchange.
6. notification-service asinhrono konzumira događaj, upisuje notifikaciju u svoju bazu i loguje je.

Ceo lanac je vidljiv kao jedan trace u Zipkin-u, uključujući i skok kroz broker.

2. Tehnologije i obrazloženje izbora

Oblast	Izbor	Zašto
Jezik i framework	Java 17, Spring Boot 4.0.5	Aktuator daje health, metrike i Prometheus endpoint bez pisanja koda
API Gateway	Spring Cloud Gateway (WebMVC)	Deklarativno rutiranje kroz konfiguraciju, bez pisanja rutera
Baza	PostgreSQL 15, Spring Data JPA	JPA uklanja SQL boilerplate za CRUD
Broker	RabbitMQ	Topic exchange omogućava dodavanje konzumera bez izmene producera
Reaktivnost	Spring WebFlux i RxJava 3	Specifikacija navodi RxJava imenom, pa je uz Flux dodat i Flowable endpoint
Metrike	Prometheus i Grafana	Micrometer izlaze metrike u Prometheus formatu bez dodatnog koda
Logovi	Loki, preko loki4j appender-a	Servisi guraju logove direktno, nema Promtail sidecar-a
Trace-ovi	Zipkin, Micrometer Tracing (Brave)	Sopstveni UI koji jasno prikazuje lanac poziva
CI/CD	GitHub Actions	Isti provajder kao repozitorijum
Statička analiza	SonarCloud sa JaCoCo	Besplatan za javne repozitorijume, sa quality gate mehanizmom
Testiranje	JUnit 5, Mockito, RestAssured, Awaitility	Awaitility je neophodan jer je notifikacija asinhrona

gRPC nije implementiran. Specifikacija ga eksplicitno označava kao opcioni, u kombinaciji sa REST-om. Sva četiri obavezna tipa komunikacije su pokrivena.

3. Ispunjenost zahteva po tačkama specifikacije

Tačka 1 - Opšti zahtevi početne aplikacije

Pet mikroservisa, uz zahtevani minimum od četiri: api-gateway, employee-service, client-service, task-service, notification-service.

employee-service i task-service imaju pun CRUD, client-service ima listanje, dohvatanje i kreiranje plus dva reaktivna stream endpointa, notification-service izlaze listu notifikacija.

Rute gateway-a su konfigurisane deklarativno:

```
spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      server:
        webmvc:
          routes:
            - id: employee-service
              uri: ${EMPLOYEE_SERVICE_URL:http://localhost:8081}
              predicates:
                - Path=/api/employees/**
              filters:
                - StripPrefix=1
            - id: client-service
              uri: ${CLIENT_SERVICE_URL:http://localhost:8082}
              predicates:
                - Path=/api/clients/**
              filters:
                - StripPrefix=1
            - id: task-service
              uri: ${TASK_SERVICE_URL:http://localhost:8083}
              predicates:
                - Path=/api/tasks/**
              filters:
                - StripPrefix=1
            - id: notification-service
              uri: ${NOTIFICATION_SERVICE_URL:http://localhost:8084}
              predicates:
                - Path=/api/notifications/**
              filters:
                - StripPrefix=1
```

backend/api-gateway/src/main/resources/application.yml

Logovi, metrike i trace-ovi se šalju iz svih pet servisa: metrike na Prometheus preko /actuator/prometheus, logovi u Loki sa traceId i spanId u svakom zapisu, trace-ovi u Zipkin sa uzorkovanjem od sto posto.

Tipovi komunikacije, sva četiri obavezna:

Tip	Implementacija
REST, sinhrono	Gateway rutira ka svim servisima
REST, servis ka servisu	task-service proverava zaposlenog kod employee-service preko RestClient-a
Message Queue	task-service objavljuje task.created, notification-service konzumira preko @RabbitListener
Reaktivno	client-service na WebFlux-u, Flux i RxJava Flowable kao Server-Sent Events

Provera zaposlenog pre kreiranja zadatka, sinhrono preko REST-a:

```
package com.uris.crmhrm.task_service;

import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;
import org.springframework.web.client.RestClient;

@Component
public class EmployeeClient {

    private final RestClient restClient;

    public EmployeeClient(RestClient.Builder restClientBuilder,
        @Value("${employee-service.url}") String employeeServiceUrl) {
        this.restClient = restClientBuilder.baseUrl(employeeServiceUrl).build();
    }

    public boolean employeeExists(Long employeeId) {
        return restClient.get()
            .uri("/employees/{id}", employeeId)
            .exchange((request, response) -> response.getStatusCode().is2xxSuccessful());
    }
}
```

backend/task-service/.../EmployeeClient.java

Objavljivanje događaja na RabbitMQ:

```
public void publishTaskCreated(Task task) {
    TaskCreatedEvent event = new TaskCreatedEvent(task.getId(), task.getTitle(), task.getEmployeeId());
    rabbitTemplate.convertAndSend(RabbitMqConfig.EXCHANGE_NAME, RabbitMqConfig.TASK_CREATED_ROUTING_KEY, event);
    log.info("Published {} event for task {}", RabbitMqConfig.TASK_CREATED_ROUTING_KEY, task.getId());
}
```

backend/task-service/.../TaskEventPublisher.java

Konzumiranje događaja u drugom servisu:

```

    @RabbitListener(queues = RabbitMqConfig.TASK_CREATED_QUEUE)
    public void onTaskCreated(TaskCreatedEvent event) {
        Notification notification = notificationService.record(
            "Kreiran je zadatak: " + event.title(), event.taskId());
        log.info("Stored notification {} for task {}", notification.getId(), event.taskId());
    }

```

backend/notification-service/.../TaskCreatedListener.java

Reaktivni endpointi, jedan preko Reactor-a i jedan preko RxJava biblioteke:

```

    @GetMapping(value = "/stream", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flux<Client> streamClients() {
        return reactiveClientService.streamAll();
    }

    @GetMapping(value = "/flowable", produces = MediaType.TEXT_EVENT_STREAM_VALUE)
    public Flowable<Client> streamClientsAsFlowable() {
        return Flowable.fromPublisher(reactiveClientService.streamAll());
    }

```

backend/client-service/.../ClientController.java

Tačka 2 - Git workflow i verzionisanje

Rad je išao isključivo kroz feature grane, uz obavezan pull request za svaku celinu. Direktnog push-a na main nije bilo nijednom. Ukupno 24 pull request-a, imenovana po konvenciji feature/*, fix/*, chore/* i docs/*. Commit poruke prate Conventional Commits format. Puna istorija je u sekciji 5.

Tačka 3 - Testiranje

Testovi su pisani paralelno sa razvojem, u istom pull request-u kao i kod koji pokrivaju.

Nivo	Broj	Alat
Jedinični i integracioni	60	JUnit 5, Mockito, Spring Boot test slice-ovi
End-to-end	13	RestAssured, Awaitility

Jedinični testovi koriste ugrađenu H2 bazu, pa CI ne mora da diže PostgreSQL. End-to-end testovi rade nad stvarno podignutim sistemom, kroz gateway.

Pokrivenost koda je 95,7 posto, odnosno sto posto ako se izuzmu main metode koje se ne mogu testirati. Svi testovi prolaze.

End-to-end provera celog lanca, sa čekanjem na asinhronu notifikaciju:

```

@Test
void createdTaskEventuallyProducesNotification() {
    String title = "E2E zadatak " + UUID.randomUUID();
    int employeeId = createEmployee();

    int taskId = given()
        .baseUrl(ServiceEndpoints.gateway())
        .contentType(ContentType.JSON)
        .body(taskPayload(title, employeeId))
        .when()
        .post("/api/tasks")
        .then()
        .statusCode(201)
        .extract().path("id");

    await().atMost(NOTIFICATION_TIMEOUT).pollInterval(POLL_INTERVAL).untilAsserted(() ->
        given()
            .baseUrl(ServiceEndpoints.gateway())
            .when()
            .get("/api/notifications")
            .then()
            .statusCode(200)
            .body("findAll { it.taskId == " + taskId + " }.message", hasItem("Kreiran je zadatak: " + title)));
}

```

e2e-tests/.../TaskLifecycleIT.java

Tačka 4 - CI pipeline

CI se pokreće automatski na svaki push i svaki pull request. Obuhvata build svih pet servisa kroz jedan aggregator POM, izvršavanje svih testova, SonarCloud analizu sa JaCoCo izveštajem o pokrivenosti, upload build artefakata, Docker build svih pet image-a paralelno kroz matrix strategiju, i end-to-end testove nad podignutim sistemom.

```

- name: Build i test svih servisa
  run: mvn -B -f backend/pom.xml verify

- name: SonarCloud Scan
  if: github.event_name == 'pull_request' || github.ref == 'refs/heads/main'
  env:
    GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
  run: mvn -B -f backend/pom.xml org.sonarsource.scanner.maven:sonar-maven-plugin:5.7.0.6970:sonar -Dsonar.qualitygate.wait=true

- name: Upload build artefakata
  uses: actions/upload-artifact@v4
  with:
    name: service-jars
    path: backend/*/target/*.jar
    if-no-files-found: error
    retention-days: 7

```

.github/workflows/ci.yml, ključni koraci

Tačka 5 - Docker kontejnerizacija

Svih pet servisa je kontejnerizovano, svaki sa sopstvenim Dockerfile-om. Build je multi-stage: build faza koristi Maven image, runtime faza samo JRE, pa finalni image ne sadrži Maven ni izvorni kod.


```
FROM maven:3.9.6-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn -B clean package -DskipTests

FROM eclipse-temurin:17-jre
RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

backend/employee-service/Dockerfile, isti obrazac za svih pet servisa

.dockerignore postoji u svakom od pet servisa i isključuje nepotrebno iz build konteksta:

```
target/
.git/
.gitignore
.gitattributes
.mvn/
mvnw
mvnw.cmd
*.md
```

backend/employee-service/.dockerignore

Environment promenljive pokrivaju port, URL-ove servisa, kredencijale baze i brokera, adrese Zipkin-a i Loki-ja. Tagovanje ide sa latest i sa SHA commit-a. Docker build je integrisan u CI i pokreće se automatski na svaki push.

Tačka 6 - Docker Compose orkestracija

Ceo sistem se podiže jednom komandom. docker - compose . yml sadži jedanaest kontejnera: pet servisa, PostgreSQL, RabbitMQ, Prometheus, Grafana, Loki i Zipkin.

- Mreža: svi kontejneri su na zajedničkoj crm_network bridge mreži i adresiraju se po imenu servisa.
- Environment varijable: izdvojene u .env, koji nije u verzionisanju, dok je .env.example commit-ovan kao šablon.
- Healthcheck: svaki servis proverava svoj /actuator/health, PostgreSQL koristi pg_isready, RabbitMQ rabbitmq-diagnostics ping.
- Volume-i: pgdata za bazu, grafana-data i prometheus-data za monitoring.

Primer definicije jednog servisa, sa healthcheck-om i čekanjem na zdrave zavisnosti:

```

task-service:
  build:
    context: ./backend/task-service
  image: ${IMAGE_REGISTRY}/task-service:${IMAGE_TAG}
  container_name: crm_task_service
  environment:
    SERVER_PORT: 8083
    ZIPKIN_ENDPOINT: http://zipkin:9411/api/v2/spans
    LOKI_URL: http://loki:3100/loki/api/v1/push
    SPRING_DATASOURCE_URL: jdbc:postgresql://postgres:5432/task_db
    SPRING_DATASOURCE_USERNAME: ${POSTGRES_USER}
    SPRING_DATASOURCE_PASSWORD: ${POSTGRES_PASSWORD}
    SPRING_RABBITMQ_HOST: rabbitmq
    SPRING_RABBITMQ_PORT: 5672
    SPRING_RABBITMQ_USERNAME: ${RABBITMQ_USER}
    SPRING_RABBITMQ_PASSWORD: ${RABBITMQ_PASSWORD}
    EMPLOYEE_SERVICE_URL: http://employee-service:8081
  ports:
    - "8083:8083"
  depends_on:
    zipkin:
      condition: service_healthy
    postgres:
      condition: service_healthy
    rabbitmq:
      condition: service_healthy
    employee-service:
      condition: service_healthy
  healthcheck:
    test: ["CMD-SHELL", "curl -f http://localhost:8083/actuator/health || exit 1"]
    interval: 10s
    timeout: 5s
    retries: 12
    start_period: 40s
  networks:
    - crm_network

```

docker-compose.yml, isečak

Tačka 7 - Statička analiza koda

SonarCloud je integrisan u CI pipeline i pokreće se nad svih pet modula kroz jedan scan nad aggregator POM-om. JaCoCo generiše izveštaj o pokrivenosti koji se prosleđuje Sonar-u.

Analiza pokriva code smells, potencijalne bugove i sigurnosne probleme. Quality gate obara build ako nije zadovoljen. Taj flag je namerno uključen tek kada je kod bio čist, da ne bi blokirao rad tokom razvoja.

```

- name: SonarCloud Scan
  if: github.event_name == 'pull_request' || github.ref == 'refs/heads/main'
  env:
    GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
  run: mvn -B -f backend/pom.xml org.sonarsource.scanner.maven:sonar-maven-plugin:5.7.0.6970:sonar -Dsonar.qualitygate.wait=true

```

.github/workflows/ci.yml, korak statičke analize

Analiza se pokreće na pull request-ovima i na main grani. Nije uključena na push proizvoljne feature grane, jer SonarCloud za taj slučaj ne autorizuje čitanje statusa quality gate-a.

Tačka 8 - Deployment

Deploy je realizovan u potpunosti lokalno, u kontejnerizovanom okruženju, sto specifikacija izričito dozvoljava.

Skripta `deploy.sh` podiže Docker aplikaciju sa bazom, čeka da svaki kontejner prijavi `healthy`, zatim proverava health endpoint svakog od pet servisa i rutiranje kroz gateway. Provere imaju retry mehanizam, jer mapiranje portova ume da zakasni za kontejnerom. Skripta vraća neuspeh ako bilo koja provera ne prođe.

```
echo "Pokrecem post-deploy health check..."
failed=0
for entry in api-gateway:8080 employee-service:8081 client-service:8082 task-service:8083 notification-service:8084; do
  name=${entry%*:}
  port=${entry##*:}
  if wait_for_ok "http://localhost:${port}/actuator/health"; then
    echo " ${name} je zdrav"
  else
    echo " ${name} nije zdrav (poslednji odgovor: $(http_status "http://localhost:${port}/actuator/health"))"
    failed=1
  fi
done
if [ "$failed" -ne 0 ]; then
  echo "Deploy nije uspeo, bar jedan servis nije zdrav."
  docker compose ps
  exit 1
fi
```

deploy.sh, provera zdravlja svih pet servisa

Tačka 9 - CD pipeline

CD je poseban workflow koji se pokreće automatski kada se pull request merge-uje u main. Prvi posao builda i objavljuje svih pet image-a na GitHub Container Registry, paralelno, tagovane sa latest i sa SHA commit-a.

```
- name: Build i push (${ matrix.service })
  run: |
    IMAGE_ID=$(echo "ghcr.io/${ github.repository_owner }/${ matrix.service }" | tr '[:upper:]' '[:lower:]')
    docker build -t "$IMAGE_ID:latest" -t "$IMAGE_ID:${ github.sha }" "backend/${ matrix.service }"
    docker push "$IMAGE_ID:latest"
    docker push "$IMAGE_ID:${ github.sha }"
```

.github/workflows/cd.yml, objava image-a sa dva taga

Drugi posao je post-deploy provera: povlači upravo objavljene image-e po SHA tagu, podiže ceo sistem u runner-u i proverava da svih pet servisa vraća 200 na health endpointu, zatim kroz gateway kreira zaposlenog i zadatak, proverava da zadatak za nepostojećeg zaposlenog vraća 404, i čeka da se notifikacija pojavi iz reda poruka.

Tačka 10 - Monitoring i observability

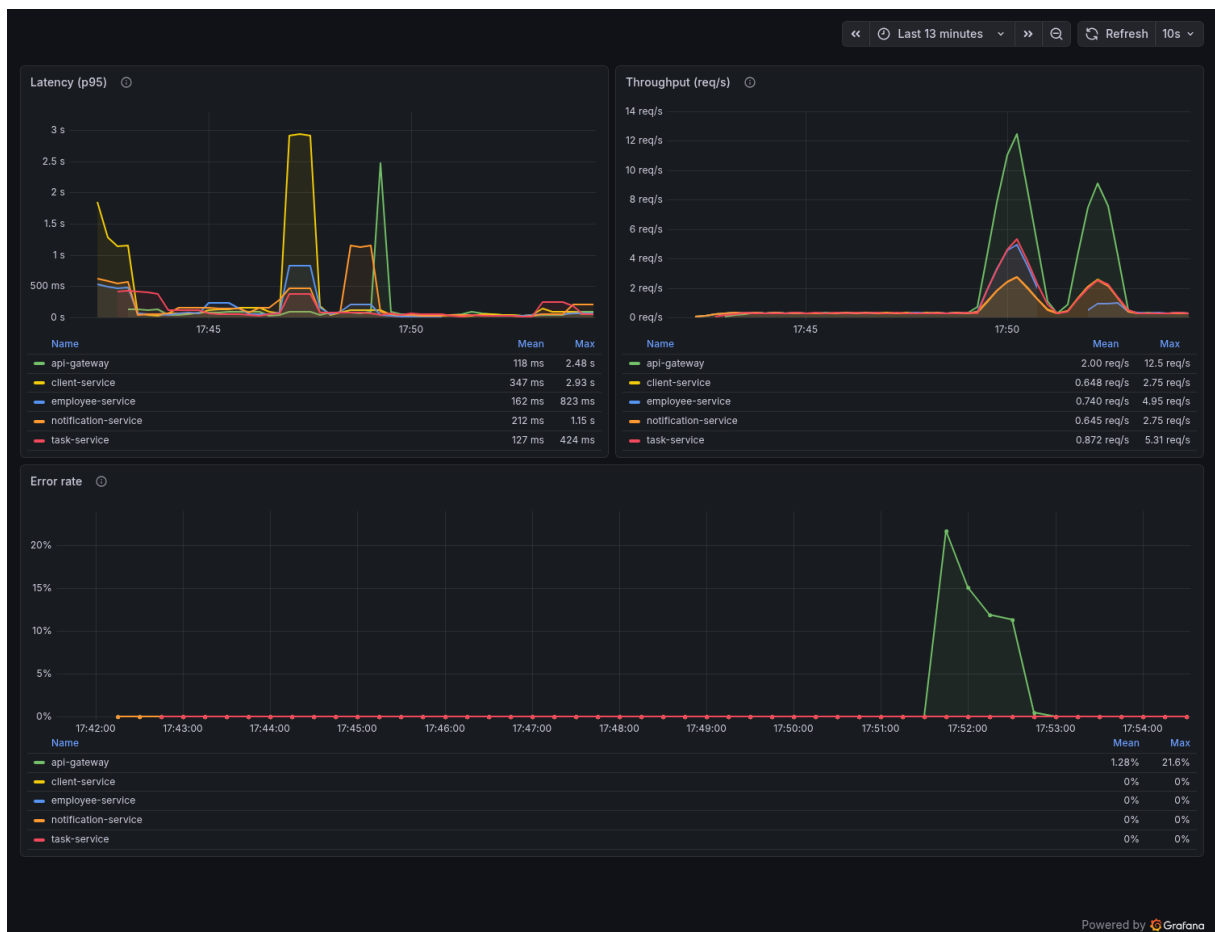
Monitoring je potpuno lokalni, zasnovan na kombinaciji Prometheus i Grafana, uz Loki za logove i Zipkin za trace-ove. Grafana se diže sa automatski provizioniranim datasource-ima i dashboardom, pa se ništa ne podešava ručno i konfiguracija preživljava `docker compose down`.

Dashboard CRM System Overview prati sve tri tražene metrike, po servisu:

Panel	Upit
Latency (p95)	<code>histogram_quantile(0.95, sum(rate(http_server_requests_seconds_bucket[1m])) by (le, application))</code>
Throughput (req/s)	<code>sum(rate(http_server_requests_seconds_count[1m])) by (application)</code>
Error rate	udeo 5xx odgovora u ukupnom saobraćaju, sa vraćanjem na nulu kada grešaka nema

Da bi Latency panel imao podatke, u svim servisima su uključeni histogram bucket-i. Bez toga Micrometer objavljuje samo zbir i broj zahteva, pa upit nad `_bucket` serijom ne bi vraćao ništa.

```
management.metrics.distribution.percentiles-histogram.http.server.requests=true
```



Grafana dashboard sa tri tražene metrike

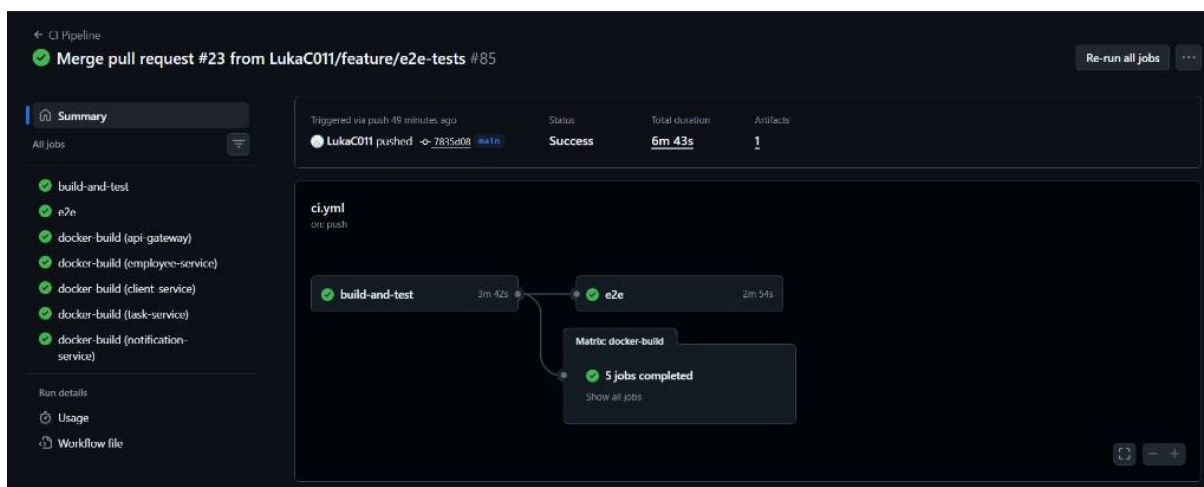
4. Tabela: zahtev do implementacije

Zahtev	Gde je implementiran	Grana i PR
Minimalno 4 mikroservisa	backend/	feature/setup-microservices, PR #2
Endpoint u svakom servisu	backend/*/.../ *Controller.java	feature/service-endpoints, PR #10
API Gateway sa rutama	api-gateway/.../ application.yml	feature/api-gateway-routing, PR #11
Baza po servisu, JPA sloj	initdb/, *Repository.java	feature/database-layer, PR #14
REST servis ka servisu	task-service/.../ EmployeeClient.java	feature/rest-communication, PR #15
Message Queue	TaskEventPublisher, TaskCreatedListener	feature/messaging-rabbitmq, PR #16
Reaktivna komunikacija	ReactiveClientService.java	feature/reactive-client-service, PR #17
Trace-ovi	management.tracing.*, zipkin u compose-u	feature/observability-tracing, PR #18
Centralizovani logovi	logback-spring.xml, loki u compose-u	feature/observability-logs, PR #19
Metrike i dashboard	prometheus.yml, grafana/dashboards/	feature/grafana-dashboards, PR #20
Unit testovi	backend/*/src/test/java/	uz svaki PR sa kodom
End-to-end testovi	e2e-tests/src/test/java/	feature/e2e-tests, PR #23
CI pipeline	.github/workflows/ci.yml	feature/ci-all-services, PR #13
Build artefakti	korak Upload build artefakata	feature/ci-all-services, PR #13
Dockerfile, multi-stage	backend/*/Dockerfile	feature/setup-microservices, PR #2
.dockerignore	backend/*/.dockerignore	chore/repo-hygiene, PR #9
Environment promenljive	.env.example, docker- compose.yml	feature/compose-full-stack, PR #12
Tagovanje image-a	.github/workflows/cd.yml	feature/cd-all-services, PR #22
Compose, mreža, volume-i	docker-compose.yml	feature/compose-full-stack, PR #12
Healthcheck mehanizmi	docker-compose.yml, curl u Dockerfile-u	PR #9 i PR #12
Statička analiza	korak SonarCloud Scan, JaCoCo u pom-ovima	PR #14 i PR #21
Deployment sa health proverom	deploy.sh	feature/cd-all-services, PR #22
CD pipeline	.github/workflows/cd.yml	PR #5 i PR #22
Post-deploy provera	job post-deploy-check	feature/cd-all-services, PR #22
Latency, throughput, error rate	grafana/dashboards/crm- overview.json	feature/grafana-dashboards, PR #20

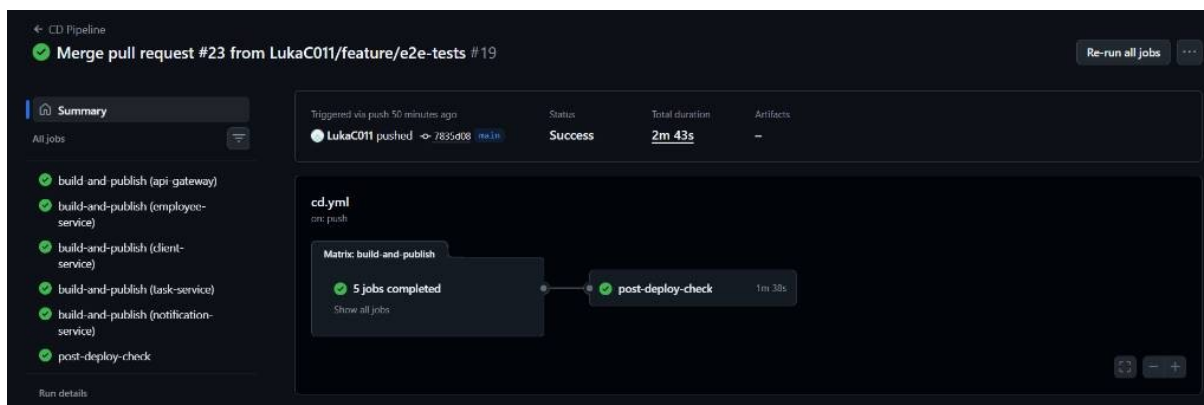
5. Git workflow i istorija pull request-ova

Svaka celina je razvijana na zasebnoj grani i uvedena u main isključivo kroz pull request. main u svakom trenutku sadrži stabilan kod, sa zelenim CI-jem.

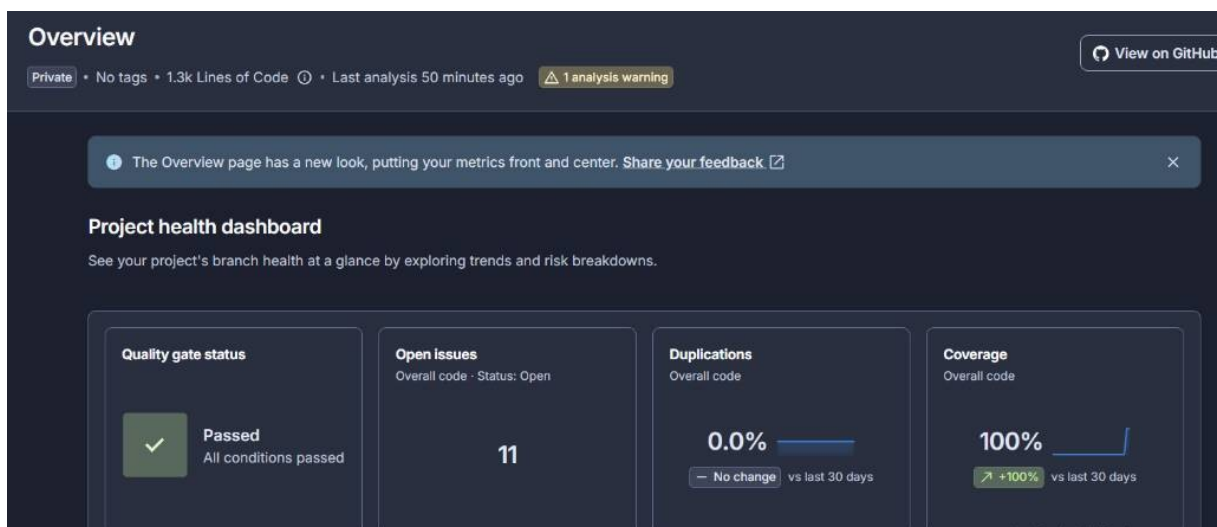
PR	Grana	Naslov
#1	feature/init-project-structure	Added base project structure and .gitignore
#2	feature/setup-microservices	feat: Initialized Spring Boot services, added DockerFile and docker-compose
#3	feature/testing-and-ci	feat: Added JUnit test and GitHub actions CI pipeline
#4	feature/static-analysis	feat: SonarCloud integration
#5	feature/cd-pipeline	feat: Added CD pipeline for GitHub packages and a local deploy script
#6	feature/monitoring	feat: Prometheus and Grafana monitoring implementation
#7	fix/grafana-config	fix: Prometheus and Grafana variable configuration
#8	chore/repo-hygiene	chore: normalize line endings
#9	chore/repo-hygiene	chore: Docker file naming and line ending hygiene
#10	feature/service-endpoints	feat: add REST endpoints and actuator metrics to every service
#11	feature/api-gateway-routing	feat: route /api traffic through the gateway to each service
#12	feature/compose-full-stack	feat: run the whole system from Docker Compose
#13	feature/ci-all-services	ci: build, test and package every service
#14	feature/database-layer	feat: add JPA persistence layer to all four business services
#15	feature/rest-communication	feat: validate employee over REST before creating a task
#16	feature/messaging-rabbitmq	feat: publish task.created events to notification service
#17	feature/reactive-client-service	feat: serve clients reactively with Reactor and RxJava streams
#18	feature/observability-tracing	feat: trace requests across all services with Zipkin
#19	feature/observability-logs	feat: ship service logs to Loki with trace correlation
#20	feature/grafana-dashboards	feat: add Grafana dashboard for latency, throughput and errors
#21	feature/sonar-quality-gate	ci: fail the build when the Sonar quality gate fails
#22	feature/cd-all-services	feat: publish all five images and verify the deployment
#23	feature/e2e-tests	test: add end to end tests against the running stack
#24	docs/project-documentation	docs: add README and project documentation



Zeleni CI pipeline nad main granom



Zeleni CD pipeline sa post-deploy proverom



SonarCloud rezultat sa pokrivenošću koda

6. Uputstvo za lokalni deploy

Jedini preduslov je Docker sa Compose dodatkom.

```
git clone https://github.com/LukaC011/devOps.git
cd devOps
cp .env.example .env
./deploy.sh
```

Skripta builda sve image-e, podiže jedanaest kontejnera, čeka da svaki prijavi healthy, proverava health endpoint svih pet servisa i rutiranje kroz gateway, pa ispisuje adrese alata. Prvo pokretanje traje nekoliko minuta jer se svi image-i grade lokalno.

Na Windows-u, ako kloniranje prijavi `Filename too long`, uključiti podršku za duge putanje jednom komandom `git config --global core.longpaths true`.

Provera da sistem stvarno radi:

```
curl -X POST http://localhost:8080/api/employees \
-H 'Content-Type: application/json' \
-d '{"firstName":"Mila","lastName":"Jovanovic","position":"HR Manager","email":"mila@crm.rs"}'

curl -X POST http://localhost:8080/api/tasks \
-H 'Content-Type: application/json' \
-d '{"title":"Pripremi ponudu","description":"Ponuda za klijenta","employeeId":1}'

curl http://localhost:8080/api/notifications
```

Poslednji poziv vraća notifikaciju koja je nastala automatski, preko RabbitMQ-a.

Alat	Adresa
Gateway	http://localhost:8080/api
Grafana	http://localhost:3000
Prometheus	http://localhost:9090
Zipkin	http://localhost:9411
RabbitMQ	http://localhost:15672

Gašenje sistema, uz brisanje podataka: `docker compose down -v`

7. Dokazi o radu sistema

Trace kroz četiri servisa

Jedan POST /api/tasks daje u Zipkin-u trace od sedam spanova, uključujući i asinhroni skok kroz RabbitMQ:

SERVIS	SPAN	KIND
api-gateway	http post /api/tasks/**	SERVER
api-gateway	http post	CLIENT
task-service	http post /tasks	SERVER
task-service	http get	CLIENT
employee-service	http get /employees/{id}	SERVER
task-service	crm.exchange/task.created send	PRODUCER
notification-service	task.created.queue receive	CONSUMER

Izlaz Zipkin API-ja za jedan trace

Korelacija logova po trace ID-u

Svi logovi jednog zahteva se izdvajaju u Grafani, nad Loki datasource-om. Isti trace ID postoji i u logovima notification-service-a, iako je do njega poruka stigla asinhrono kroz broker:

```
{application="task-service"} | traceId=`6a8ca48c49c8a1434e4491a9391de324`  
  
task-service          1 zapis  
  2026-08-24 20:07:41.307 INFO [traceId=6a8ca48c...,spanId=3b363d18...]  
TaskEventPublisher  
notification-service  1 zapis  
  2026-08-24 20:07:41.885 INFO [traceId=6a8ca48c...,spanId=505cbe46...]  
TaskCreatedListener
```

Post-deploy provera u CD-u

Izlaz post-deploy-check job-a, nad image-ima povučenim sa registry-ja po SHA tagu:

```
IMAGE_TAG: 6bf76ed383f788bbc695a7cb3584023551c3f43d  
  
api-gateway          -> 200  
employee-service     -> 200  
client-service       -> 200  
task-service         -> 200  
notification-service -> 200  
  
POST /api/tasks                -> 201  
POST /api/tasks sa nepostojećim zaposlenim -> 404  
notifikacija stigla iz reda: [{"message":"Kreiran je zadatak: Smoke  
task","taskId":1}]
```

Verifikacija iz čistog klona

Sistem je proveren iz svežeg klona repozitorijuma, na praznim volume-ima, sto je jedini način da se potvrdi da init skripta stvarno kreira četiri baze:

```
svež git clone -> cp .env.example .env -> ./deploy.sh  
  
11 kontejnera healthy
```

```
svih 5 servisa /actuator/health -> 200
rutiranje kroz gateway -> 200
baze kreirane: client_db, employee_db, notification_db, task_db
Grafana: datasource-i Prometheus i Loki, dashboard CRM System Overview
Prometheus: 5/5 targeta up
E2E: 13/13 testova prolazi
```

Svi kontejneri u radu

NAME	STATUS
crm_api_gateway	Up (healthy)
crm_client_service	Up (healthy)
crm_employee_service	Up (healthy)
crm_grafana	Up (healthy)
crm_loki	Up
crm_notification_service	Up (healthy)
crm_postgres	Up (healthy)
crm_prometheus	Up (healthy)
crm_rabbitmq	Up (healthy)
crm_task_service	Up (healthy)
crm_zipkin	Up (healthy)

docker compose ps